

Automatisierter Testbericht

Projekt: Inventory Management System

Testframework: pytest

Verwendeter Befehl: python -m pytest app/tests -v

Testergebnis:

10 Tests gesammelt

10 Tests bestanden

0 fehlgeschlagen

0 Fehler

Zusammenfassung: Die automatisierten Tests überprüfen die Kernlogik des Inventory-Management-Systems. Die Tests decken Lagerbewegungen, Produkterstellung/-löschung, Validierungsfehler sowie die Benutzererstellung einschließlich doppelter Benutzername- und E-Mail-Prüfungen ab.

Testkategorien:

- Inventory-Service-Tests
- Product-Service-Tests
- User-Service-Tests

Verwendete Testdaten: Für jeden Test wird mithilfe von pytest-Fixtures eine saubere In-Memory-SQLite-Datenbank erstellt. Die Basisdaten enthalten eine Kategorie, einen Lagerort, einen Admin-Benutzer und ein Produkt.

Endergebnis: Alle automatisierten Tests wurden erfolgreich bestanden.

Übersicht der automatisierten Testfälle

Testfall-ID	Beschreibung	Erwartetes Ergebnis	Tatsächliches Ergebnis	Status
TC_001	Wareneingang erhöht die Produktmenge	Produktmenge wird korrekt erhöht	Bestanden	Bestanden
TC_002	Warenausgang reduziert die Produktmenge	Produktmenge wird korrekt reduziert	Bestanden	Bestanden
TC_003	Warenausgang größer als verfügbarer Bestand	ValueError wird ausgelöst	Bestanden	Bestanden
TC_004	Produkte mit niedrigem Bestand werden erkannt	Produkt erscheint in der Low-Stock-Liste	Bestanden	Bestanden
TC_005	Produkt erfolgreich erstellen	Produkt wird erfolgreich erstellt	Bestanden	Bestanden
TC_006	Leerer Produktname	ValueError wird ausgelöst	Bestanden	Bestanden

TC_007	Produkt löschen	Produkt wird erfolgreich entfernt	Bestanden	Bestanden
TC_008	Benutzer erfolgreich erstellen	Benutzer wird erfolgreich erstellt	Bestanden	Bestanden
TC_009	Doppelte E-Mail-Adresse	ValueError wird ausgelöst	Bestanden	Bestanden
TC_010	Doppelter Benutzername	ValueError wird ausgelöst	Bestanden	Bestanden

Testfälle – Inventory Management System

TC_001 – Wareneingang erhöht die Produktmenge

Feld	Details
Testfall-ID	TC_001
Titel/Beschreibung	Überprüfung, ob der Wareneingang die Produktmenge korrekt erhöht.
Voraussetzungen	Eine saubere Testdatenbank ist vorhanden. Ein Produkt mit Menge 10 existiert. Ein Benutzer existiert ebenfalls.
Testschritte	1. InventoryService erstellen. 2. Vorhandenes Produkt und Benutzer auswählen. 3. <code>stock_in(product.id, user.id, 3, note="Restock")</code> ausführen. 4. Produktmenge überprüfen.
Testdaten/Eingaben	Produkt: Laptop, Anfangsmenge: 10, Wareneingang: 3, Benutzer: admin
Erwartetes Ergebnis	Die Produktmenge erhöht sich von 10 auf 13. Ein Stock-Movement-Eintrag wird erstellt.
Tatsächliches Ergebnis	Die Produktmenge wurde korrekt erhöht und die Bewegung wurde gespeichert.
Status	Bestanden
Kommentare	Keine Probleme festgestellt.

TC_002 – Warenausgang reduziert die Produktmenge

Feld	Details
Testfall-ID	TC_002
Titel/Beschreibung	Überprüfung, ob der Warenausgang die Produktmenge korrekt reduziert.
Voraussetzungen	Eine saubere Testdatenbank ist vorhanden. Ein Produkt mit Menge 10 existiert.
Testschritte	1. InventoryService erstellen. 2. Vorhandenes Produkt und Benutzer auswählen. 3. <code>stock_out(product.id, user.id, 4, note="Shipment")</code> ausführen. 4. Produktmenge überprüfen.
Testdaten/Eingaben	Produkt: Laptop, Anfangsmenge: 10, Warenausgang: 4
Erwartetes Ergebnis	Die Produktmenge reduziert sich von 10 auf 6.
Tatsächliches Ergebnis	Die Produktmenge wurde korrekt reduziert.
Status	Bestanden
Kommentare	Keine Probleme festgestellt.

TC_003 – Warenausgang größer als verfügbarer Bestand

Feld	Details
------	---------

Testfall-ID	TC_003
Titel/Beschreibung	Überprüfung, ob das System einen zu großen Warenausgang verhindert.
Voraussetzungen	Ein Produkt mit Menge 10 existiert.
Testschritte	1. InventoryService erstellen. 2. <code>stock_out(product.id, user.id, 999)</code> ausführen. 3. Fehler überprüfen.
Testdaten/Eingaben	Verfügbare Menge: 10, gewünschter Warenausgang: 999
Erwartetes Ergebnis	Das System löst einen <code>ValueError</code> aus.
Tatsächliches Ergebnis	Der Fehler wurde korrekt ausgelöst.
Status	Bestanden
Kommentare	Grenzfall erfolgreich getestet.

TC_004 – Produkte mit niedrigem Bestand werden erkannt

Feld	Details
Testfall-ID	TC_004
Titel/Beschreibung	Überprüfung, ob Produkte mit niedrigem Bestand korrekt erkannt werden.
Voraussetzungen	Ein Produkt mit Menge 10 und Mindestmenge 2 existiert.
Testschritte	1. InventoryService erstellen. 2. Warenausgang von 8 Einheiten durchführen. 3. Low-Stock-Funktion ausführen.
Testdaten/Eingaben	Produkt: Laptop, Menge: 10, Mindestmenge: 2
Erwartetes Ergebnis	Das Produkt erscheint in der Low-Stock-Liste.
Tatsächliches Ergebnis	Das Produkt wurde korrekt erkannt.
Status	Bestanden
Kommentare	Low-Stock-Logik funktioniert korrekt.

TC_005 – Produkt erfolgreich erstellen

Feld	Details
Testfall-ID	TC_005
Titel/Beschreibung	Überprüfung, ob ein neues Produkt erfolgreich erstellt werden kann.
Voraussetzungen	Eine gültige Kategorie und ein gültiger Lagerort existieren.
Testschritte	1. ProductService erstellen. 2. <code>create_product()</code> mit gültigen Daten ausführen. 3. Produkt speichern.
Testdaten/Eingaben	Name: Keyboard, Beschreibung: Mechanical keyboard, Menge: 5
Erwartetes Ergebnis	Das Produkt wird erfolgreich erstellt und gespeichert.
Tatsächliches Ergebnis	Das Produkt wurde erfolgreich erstellt.
Status	Bestanden
Kommentare	Standardfall erfolgreich getestet.

TC_006 – Leerer Produktname

Feld	Details
Testfall-ID	TC_006
Titel/Beschreibung	Überprüfung, ob das System einen leeren Produktnamen ablehnt.
Voraussetzungen	ProductService ist verfügbar.

Testschritte	1. ProductService erstellen. 2. <code>create_product()</code> mit leerem Namen ausführen.
Testdaten/Eingaben	Produktname: leer
Erwartetes Ergebnis	Das System löst einen <code>ValueError</code> aus.
Tatsächliches Ergebnis	Der Fehler wurde korrekt ausgelöst.
Status	Bestanden
Kommentare	Validierung funktioniert korrekt.

TC_007 – Produkt löschen

Feld	Details
Testfall-ID	TC 007
Titel/Beschreibung	Überprüfung, ob ein Produkt erfolgreich gelöscht werden kann.
Voraussetzungen	Ein Produkt existiert in der Datenbank.
Testschritte	1. ProductService erstellen. 2. Produkt löschen. 3. Überprüfen, ob das Produkt entfernt wurde.
Testdaten/Eingaben	Produkt: Laptop
Erwartetes Ergebnis	Das Produkt wird erfolgreich entfernt.
Tatsächliches Ergebnis	Das Produkt wurde erfolgreich gelöscht.
Status	Bestanden
Kommentare	Löschfunktion funktioniert korrekt.

TC_008 – Benutzer erfolgreich erstellen

Feld	Details
Testfall-ID	TC 008
Titel/Beschreibung	Überprüfung, ob ein Benutzer erfolgreich erstellt werden kann.
Voraussetzungen	Kein Benutzer mit derselben E-Mail oder demselben Benutzernamen existiert.
Testschritte	1. UserService erstellen. 2. <code>create_user()</code> mit gültigen Daten ausführen.
Testdaten/Eingaben	Benutzername: <code>employee1</code> , E-Mail: employee1@example.com
Erwartetes Ergebnis	Benutzer wird erfolgreich erstellt.
Tatsächliches Ergebnis	Benutzer wurde erfolgreich erstellt.
Status	Bestanden
Kommentare	Benutzererstellung funktioniert korrekt.

TC_009 – Doppelte E-Mail-Adresse

Feld	Details
Testfall-ID	TC 009
Titel/Beschreibung	Überprüfung, ob doppelte E-Mail-Adressen verhindert werden.
Voraussetzungen	Ein Benutzer mit derselben E-Mail existiert bereits.
Testschritte	1. Ersten Benutzer erstellen. 2. Zweiten Benutzer mit gleicher E-Mail erstellen.
Testdaten/Eingaben	E-Mail: employee1@example.com
Erwartetes Ergebnis	Das System löst einen <code>ValueError</code> aus.

Tatsächliches Ergebnis	Der Fehler wurde korrekt ausgelöst.
Status	Bestanden
Kommentare	Validierung funktioniert korrekt.

TC_010 – Doppelter Benutzername

Feld	Details
Testfall-ID	TC_010
Titel/Beschreibung	Überprüfung, ob doppelte Benutzernamen verhindert werden.
Voraussetzungen	Ein Benutzer mit demselben Benutzernamen existiert bereits.
Testschritte	1. Ersten Benutzer erstellen. 2. Zweiten Benutzer mit gleichem Benutzernamen erstellen.
Testdaten/Eingaben	Benutzername: employee1
Erwartetes Ergebnis	Das System löst einen <code>ValueError</code> aus.
Tatsächliches Ergebnis	Der Fehler wurde korrekt ausgelöst.
Status	Bestanden
Kommentare	Validierung funktioniert korrekt.

Testergebnis

```

1 file changed, 3 insertions(+), 3 deletions(-)
PS C:\Users\aadat\Desktop\inventory-management-fhnw> python -m pytest app/tests -v
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\aadat\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\aadat\Desktop\inventory-management-fhnw
configfile: pytest.ini
plugins: anyio-4.12.1, cov-7.1.0
collected 10 items

app/tests/test_inventory_service.py::test_stock_in_increases_quantity PASSED [ 10%]
app/tests/test_inventory_service.py::test_stock_out_decreases_quantity PASSED [ 20%]
app/tests/test_inventory_service.py::test_stock_out_more_than_available_raises_error PASSED [ 30%]
app/tests/test_inventory_service.py::test_get_low_stock_products_returns_expected_items PASSED [ 40%]
app/tests/test_product_service.py::test_create_product_success PASSED [ 50%]
app/tests/test_product_service.py::test_create_product_with_empty_name_raises_error PASSED [ 60%]
app/tests/test_product_service.py::test_delete_product_removes_record PASSED [ 70%]
app/tests/test_user_service.py::test_create_user_success PASSED [ 80%]
app/tests/test_user_service.py::test_create_user_duplicate_email_raises_error PASSED [ 90%]
app/tests/test_user_service.py::test_create_user_duplicate_username_raises_error PASSED [100%]

===== 10 passed in 0.20s =====
PS C:\Users\aadat\Desktop\inventory-management-fhnw>

```

Alle automatisierten Tests wurden erfolgreich mit pytest ausgeführt.